

3.1.加减

p.1

减法就是加上负数的补码

3.1.1.更快的进位模式

常规串行模式是进位由低到高传递，高位要等低位，速度慢
而不管哪一个高位，实际上都嵌套依赖于最低位

SUM

硬件差异

- V1: ALU-乘积-乘数为 2:2:1
- V2: ALU-乘积-乘数为 1:2:1
- V3: 没有右侧的乘数寄存器
- 除法完全一致

乘法算法

对 $M \times P$:

① V1:

$$\left[\begin{array}{l} \text{if } P_0 = 1 \left[\begin{array}{l} \text{result} + = M \\ M \ll 1 \\ P \gg 1 \end{array} \right. \end{array} \right.$$

② V2/V3:

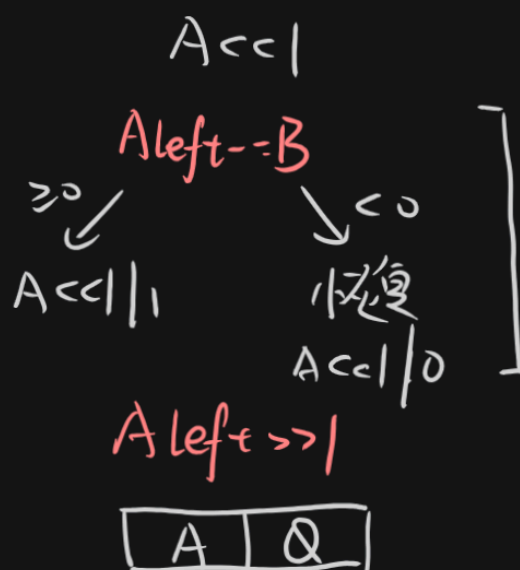
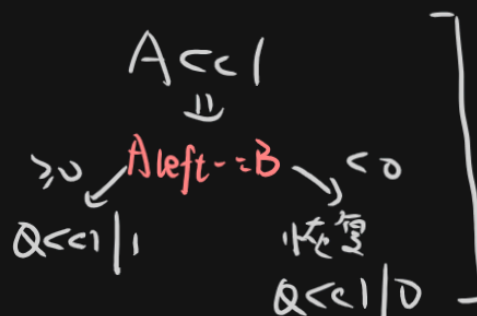
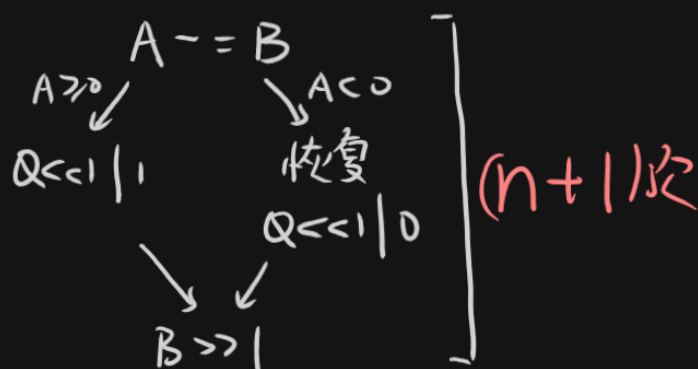
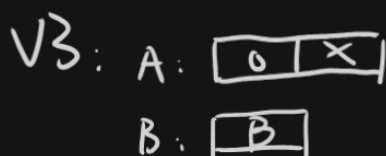
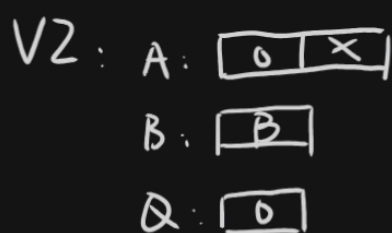
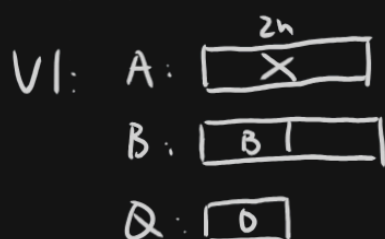
Full P

high	low
0	P

$$\left[\begin{array}{l} \text{if } P_0 = 1 \left[\begin{array}{l} P_{\text{high}} + = M \\ \text{Full P} \gg 1 \end{array} \right. \end{array} \right.$$

除法算法

$$X / B = Q \dots A$$



注意除法V1的B很特殊，需要**高位对齐**，比如 $B=2=0010\ 0000$ ，且V1需要操作 $n+1$ 个循环

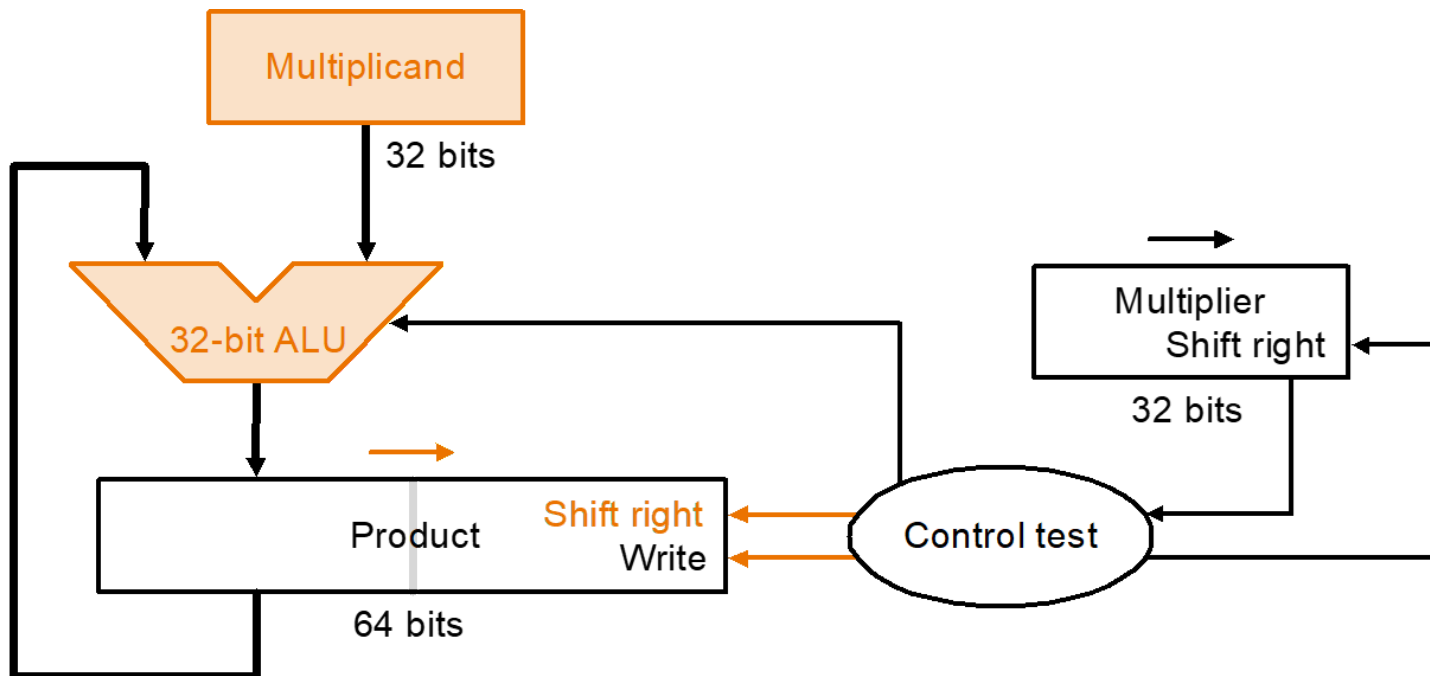
3.2.乘法

基本：移位相加法(V1)

Error: spawn pdf2svg ENOENT

简而言之：对 $M * P$ 的运算，如果 $P_0 = 1$, $result += M$ ；然后 $M \ll 1, P \gg 1$ ，进入下一个循环。循环停止条件：乘数为0或达到次数限制（比如上图的32次）

改进版乘法器V2：



相比V1的硬件改进：

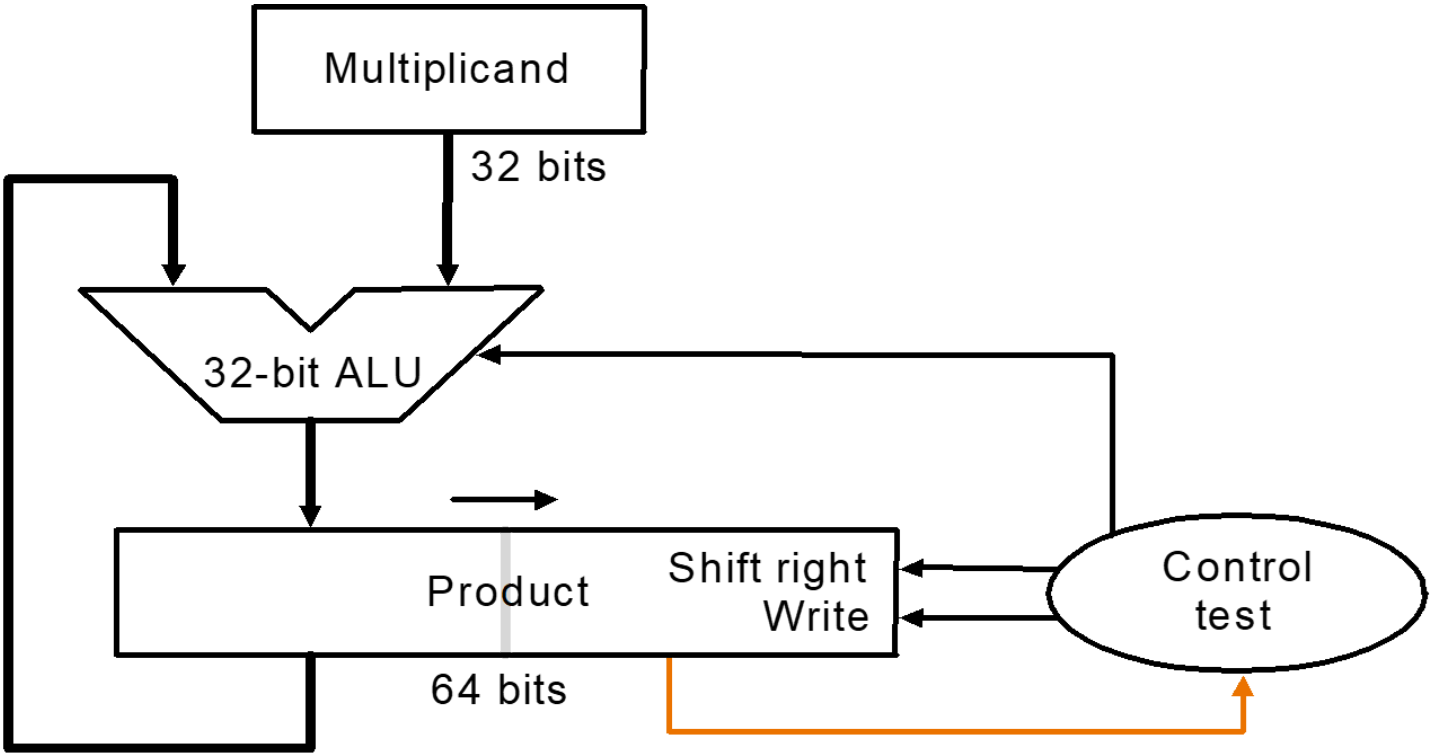
- ALU：64位->32位
- 64位积寄存器->CF+32位积+32位乘数
- 被乘数不再左移

把乘数放在部分积寄存器的低半，若最低位为1就把被乘数加到部分积的高半，然后整体右移，重复位数次，最终部分积就是乘积。

具体步骤（假设 n 位）：

1. 初始化：寄存器 M = 被乘数， $P = (P_{high} | P_{low})$ ，令 $P_{high} = 0$ ， $P_{low} =$ 乘数 Q 。
2. 对 $i = 1$ 到 n 重复：
 - 检查 P 的最低位 P_0 （即当前乘数最低位）。
 - 如果 $P_0 = 1$ ，则执行 $P_{high} += M$ （把被乘数加到部分积高半）。
 - 整体将 P 右移 1 位（高半和低半一起右移，丢弃最低位，最高位填入0）。
3. 循环结束后， P （高半与低半合并）就是最终乘积。

改进版乘法器V3



运算方法上和V2是一样的，但是硬件构成不同：

- 没有乘数寄存器

版本	ALU/加法器宽度	寄存器组织	移位对象
V1	64 位	被乘数寄存器 + 乘数寄存器 + 64 位结果寄存器	M 左移，P 右移
V2	32 位	CF + 部分积高半 + 乘数低半，合并成一组部分积/乘数寄存器	整个 P 统一右移
V3	32 位	无独立乘数寄存器，乘数直接放在部分积寄存器低半	整个部分积寄存器右移

3.3.除法

通用符号（本节统一约定）

- B：除数寄存器
- Q：商寄存器（Quotient）（V3 可不需要独立 Q）
- A：余数/部分余数寄存器（Remainder / Partial Remainder）
 - 在 V2/V3 中，常把 A 视为 2n bit，并分为：
 - A_left：A 的高 n 位（部分余数）
 - A_right：A 的低 n 位（用来移入被除数位/或累积商位）
 - 判断“是否为负”：看 A 或 A_left 的符号位（MSB）。

V1：恢复余数法

每轮先做 $A = A - B$ ，根据正负决定商位；然后 B 右移 1 位。Q 在分支里左移写入 1/0。

初始化

- **A**: 2n位，放被除数
- **Q**: n位，清零
- **B**: 2n位，放除数，高位对齐
- **计数器**: 重复 n+1 次

主循环（重复 n 次）

Step 1: 试减

- $A = A - B$

Step 2: 判断并写商位

- 若 $A \geq 0$
 - $Q = (Q \ll 1) \mid 1$
- 若 $A < 0$
 - 恢复: $A = A + B$
 - $Q = (Q \ll 1) \mid 0$

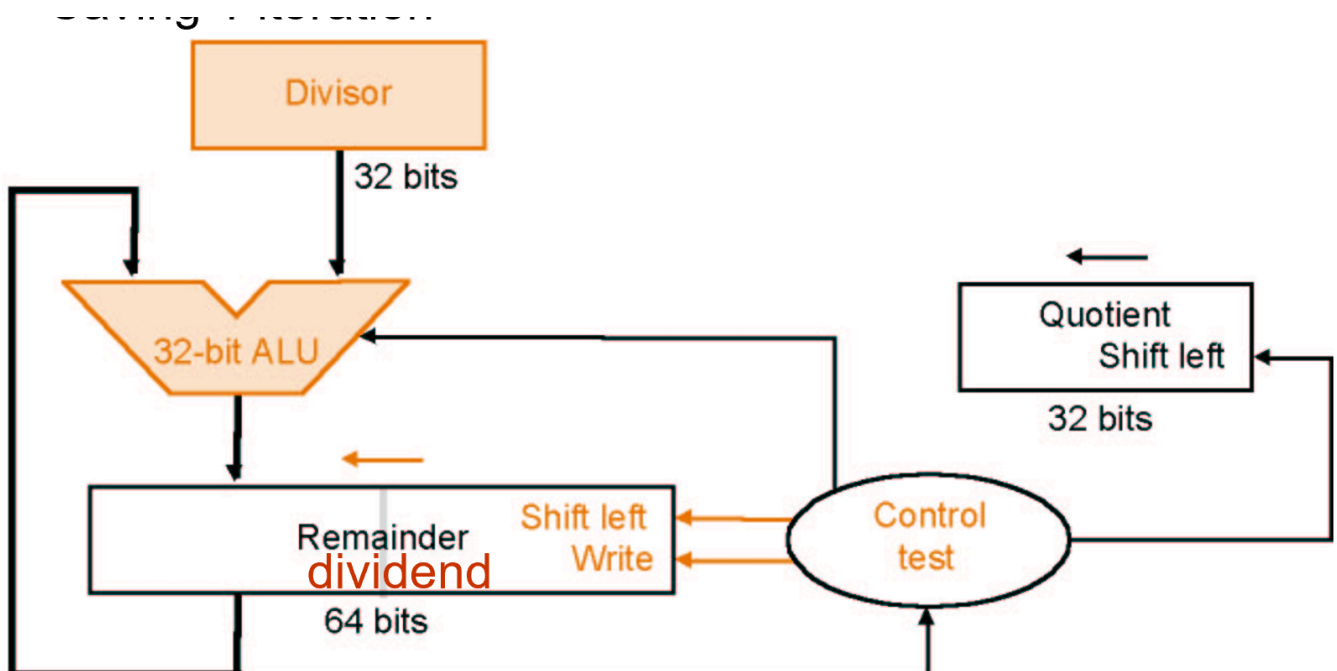
Step 3: 除数右移

- $B = B \gg 1$

一句话总结

A 试减 B，成功商写 1，失败恢复商写 0；然后把 B 右移对齐下一位。

V2:



每轮先左移 A；然后只对 A_left 做 $-B$ ；商位写入 Q。

和V1的硬件差别

- ALU: 64位->32位
- 64位余数寄存器A->CF+32位余数+32位商
- 被除数不再左移

初始化（按图2流程对应）

- A (2n bit)： $A = [0 \mid \text{被除数}]$ （即 $A_{\text{left}} = 0$, $A_{\text{right}} = \text{被除数}$ ）
- Q (n bit)： $Q = 0$
- B (n bit)： $B = \text{除数}$
- 循环次数： n

主循环（重复 n 次）

Step 1: A 左移 1 位

- $A = A \ll 1$

Step 2: 对 A_left 试减

- $A_{\text{left}} = A_{\text{left}} - B$

Step 3: 判断并更新 Q

- 若 $A_{\text{left}} \geq 0$ ： $Q = (Q \ll 1) \mid 1$
- 若 $A_{\text{left}} < 0$ ：恢复 $A_{\text{left}} = A_{\text{left}} + B$, 并写 $Q = (Q \ll 1) \mid 0$

一句话总结

先移 A（把下一位带入左半），再用左半试减 B；成则 Q 写 1，败则恢复并写 0。

V3:

Error: spawn pdf2svg ENOENT

每轮把 1/0 直接塞进 A 的最低位；最后还要对余数做一次修正。

和V2的硬件区分

- 没有商寄存器Q

初始化

- A (2n bit)： $A = [0 \mid \text{被除数}]$
- B (n bit)： $B = \text{除数}$
- Q: 不需要独立 Q（商最终在 A 的低 n 位）
- 循环次数： n

Step 1: A 左移 1 位

- $A = A \ll 1$, LSB补0

主循环（step1不在循环中）

Step 2: 对 A_left 试减

- $A_{\text{left}} = A_{\text{left}} - B$

Step 3: 根据正负，把商位写入 A 的最低位（通过再左移一次实现）

- 若 $A_{\text{left}} \geq 0$ ： $A = (A \ll 1) \mid 1$
- 若 $A_{\text{left}} < 0$ ：恢复 $A_{\text{left}} = A_{\text{left}} + B$, 并令 $A = (A \ll 1) \mid 0$
- 循环结束，回到step2，结束修正不在循环体内

结束修正

- $A_left = A_left \gg 1$
最终结果怎么读
 - **Quotient (商)**: A_right (A 的低 n 位)
 - **Remainder (余数)**: 修正后的 A_left
- 一句话总结

A 左移→左半试减→再左移并把 1/0 写进 A 最低位；最后把 A_left 右移 1 位得到正确余数。

4.Booth算法

计算带符号二进制乘法

具体步骤

- 初始化: M被乘数, Q乘数, A累加器初始为0, 上述三者都是 n 位; Q_{-1} 1位, 初始化为0
 - Booth 算法每次看两位:
 - q_0 : 当前位
 - q_{-1} : 上一位 (初始设为 0)
- 根据这两位决定操作:

$q_0 \ q_{-1}$	含义	操作
00	0 串内部	不操作
01	一个 1 串的开始	$A+=M$
10	一个 1 串的内部	$A-=M$
11	1 串内部	不操作

然后 (A,Q,Q-1) 整体算术右移一位(最高位保持)
循环 n 次, AQ就是完整结果

5.补码加法

^066fa4

补码加法的特点是:

- 正数、负数都用**补码**表示;
 - 加法时不用单独管符号位;
 - **符号位和数值位一起参加运算**;
 - 最后如果最高位产生进位, 通常直接丢掉。
- 例如用 4 位补码:

3 = 0011

-2 = 1110

相加:

0011

+1110

=0001=1 忽略最高位进位